Solving Perfect Multicollinearity with Nested Geographic Variables in R

Paul Mosquera

Contents

1 The Problem: Nested Geographic Variables in Regression **2**
1.1 The Mathematical Reason . . . 2
1.2 The Dataset . . . 2
1.3 The Hierarchical Structure . . . 3
1.4 Demonstrating the Failure . . . 4

2 Diagnosis: Confirming the Problem **5**
2.1 Variance Inflation Factor (VIF) . . . 5
2.2 Rank of the Design Matrix . . . 5

3 The Solution **6**
3.1 Step 1: Choosing the BASE Province . . . 6
3.2 Why Does This Work? . . . 7
3.3 Step 2: Applying Sum-to-Zero Contrasts . . . 8
3.4 Fitting the Model . . . 8

4 Verification: Did the Solution Work? **9**
4.1 VIF After the Fix . . . 9
4.2 Rank of the New Design Matrix . . . 9
4.3 Visual Comparison of VIF . . . 9

5 Interpretation: Reading Coefficients Under Sum-to-Zero Contrasts **10**
5.1 The Intercept . . . 10
5.2 Province Coefficients . . . 10
5.3 A Concrete Example . . . 12

6 Applicability: Where Else Does This Problem Appear? **13**
6.1 The General Recipe . . . 13
6.2 Summary . . . 13

1 The Problem: Nested Geographic Variables in Regression

Imagine you are building a model to explain housing prices across municipalities in Belgium. Your dataset contains two geographic variables: the administrative region each municipality belongs to (AdminArea), and its province (Province). Naturally, you want to include both because regions capture broad geographic effects and provinces capture finer local differences.

The problem is that these two variables are not independent. Every province belongs to exactly one region, always. Hainaut is always in Waals_Gewest. Limburg is always in Vlaams_Gewest. There are no exceptions. This rigid hierarchical structure means that once you know the province of a municipality, you already know its region with 100% certainty.

When you try to include both variables in a regression model, R cannot distinguish the effect of the region from the effect of the province. The model literally cannot be estimated. This is called perfect multicollinearity.

1.1 The Mathematical Reason

In linear regression, the coefficients are estimated using the formula:

$$\hat{\beta} = (X'X)^{-1}X'y$$

where X is the design matrix containing all predictor variables. For this formula to work, the matrix $X'X$ must be invertible, which requires that no column of X can be written as an exact linear combination of other columns.

When Province is nested within AdminArea, the region column in X is completely determined by the province columns:

$$\det(X'X) = 0 \implies (X'X)^{-1} \text{ does not exist}$$

If you sum the province dummy columns belonging to Vlaams_Gewest, you get exactly the AdminArea dummy column:

$$x_{\text{Antwerpen}} + x_{\text{Limburg}} + x_{\text{Oost_Vl}} + \cdots = x_{\text{Vlaams_Gewest}}$$

1.2 The Dataset

We use housing price data from Belgium with 566 municipalities, focusing on two administrative regions: Vlaams_Gewest (Flemish Region) and Waals_Gewest (Walloon Region). Each municipality belongs to exactly one province, which belongs to exactly one region.

```r
path <- paste0("C:/Users/paulm/Desktop/Projects/", "nested-geographic-variables-multicollinearity/",
    "data/HousePrices.txt")
df <- read.table(path, header = TRUE)

df <- df |>
    filter(AdminArea %in% c("Vlaams_Gewest", "Waals_Gewest"))

df$AdminArea <- factor(df$AdminArea, levels = c("Vlaams_Gewest",
    "Waals_Gewest"))
df$Province <- droplevels(factor(df$Province))

cat("Rows after filter:", nrow(df), "\n")
```

```
## Rows after filter: 566
```

```r
cat("AdminArea levels:", levels(df$AdminArea), "\n")
```

```
## AdminArea levels: Vlaams_Gewest Waals_Gewest
```

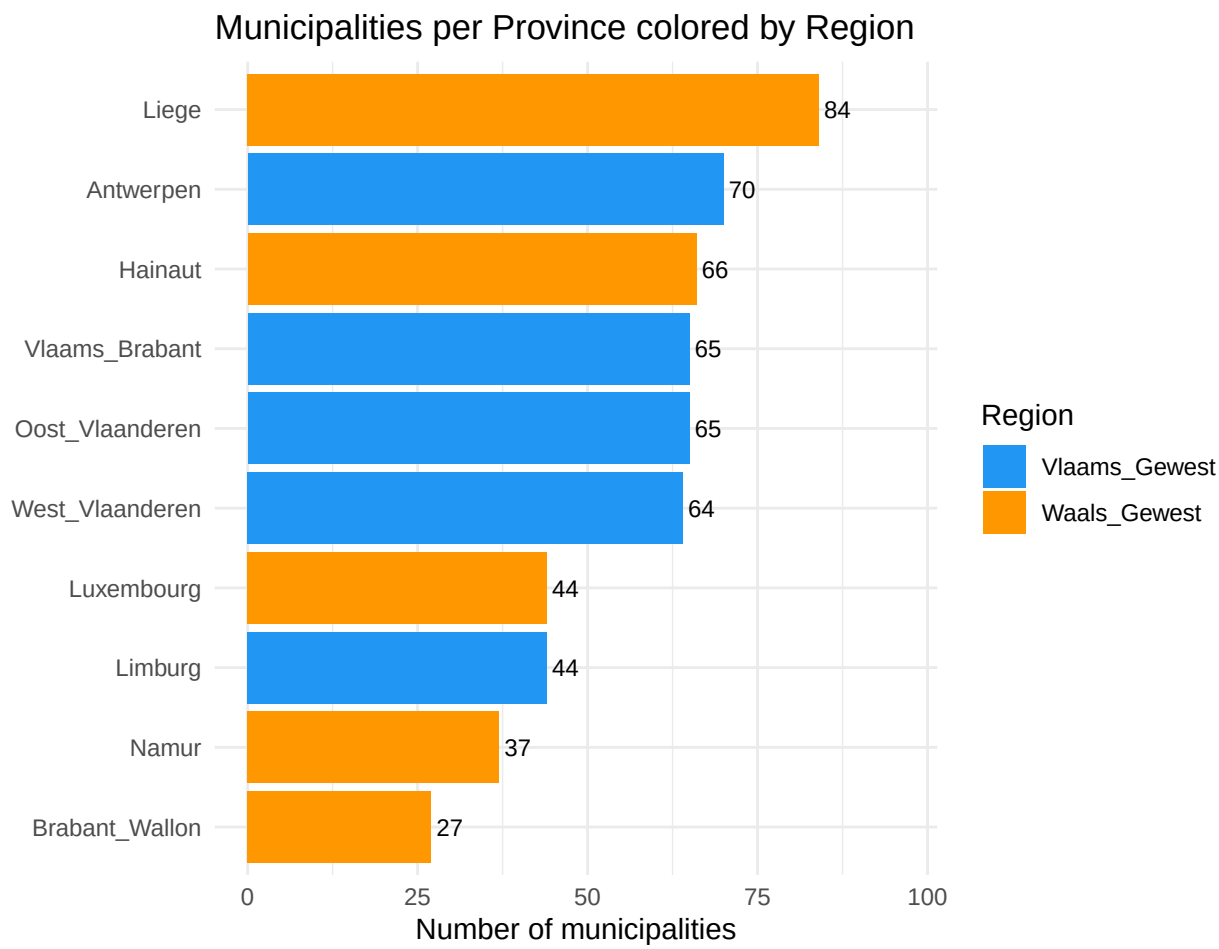
```r
cat("Province levels:", levels(df$Province), "\n")
```

```
## Province levels: Antwerpen Brabant_Wallon Hainaut Liege Limburg Luxembourg Namur Oost_Vlaanderen Vla
```

1.3 The Hierarchical Structure

Before attempting any model, it helps to visualize exactly how provinces are nested within regions. This makes the source of the problem immediately obvious.

```r
hierarchy <- df |>
    count(AdminArea, Province) |>
    arrange(AdminArea, desc(n))

ggplot(hierarchy, aes(x = reorder(Province, n), y = n, fill = AdminArea)) +
    geom_col() + geom_text(aes(label = n), hjust = -0.2,
    size = 3) + coord_flip() + scale_fill_manual(values = c(Vlaams_Gewest = "#2196F3",
    Waals_Gewest = "#FF9800")) + labs(title = "Municipalities per Province colored by Region",
    x = NULL, y = "Number of municipalities", fill = "Region") +
    theme_minimal(base_size = 11) + expand_limits(y = max(hierarchy$n) *
    1.15)
```



The color pattern is the key insight: every province bar is a single color. There is no province that is split between two regions. This perfect separation is precisely what causes the model to fail.

1.4 Demonstrating the Failure

```r
model_naive <- lm(med_price_house ~ AdminArea + Province,
    data = df)
summary(model_naive)
```

```
##
## Call:
## lm(formula = med_price_house ~ AdminArea + Province, data = df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -99.755 -20.920  -3.104  19.381 163.409
##
## Coefficients: (1 not defined because of singularities)
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)               240.585      4.068  59.139  < 2e-16 ***
## AdminAreaWaals_Gewest     -85.558      6.918 -12.367  < 2e-16 ***
## ProvinceBrabant_Wallon    101.564      8.615  11.789  < 2e-16 ***
## ProvinceHainaut           -20.372      6.990  -2.914 0.003708 **
```

```
## ProvinceLiege             17.596      6.716   2.620 0.009032 **
## ProvinceLimburg          -55.620      6.548  -8.494  < 2e-16 ***
## ProvinceLuxembourg        -4.107      7.592  -0.541 0.588793
## ProvinceNamur                 NA         NA      NA       NA
## ProvinceOost_Vlaanderen  -21.225      5.863  -3.620 0.000321 ***
## ProvinceVlaams_Brabant    13.784      5.863   2.351 0.019070 *
## ProvinceWest_Vlaanderen  -38.830      5.887  -6.596 9.82e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 34.04 on 556 degrees of freedom
## Multiple R-squared:  0.5955, Adjusted R-squared:  0.5889
## F-statistic: 90.94 on 9 and 556 DF,  p-value: < 2.2e-16
```

Look carefully at the output. Several province coefficients appear as NA. This is R silently telling you that it cannot estimate those coefficients because the information they would carry is perfectly confounded by the AdminArea variable.

2 Diagnosis: Confirming the Problem

Seeing NA coefficients in the output is already a clear symptom, but as an analyst you want to quantify the problem formally. There are two standard tools for this: the Variance Inflation Factor (VIF) and inspecting the rank of the design matrix.

2.1 Variance Inflation Factor (VIF)

The VIF measures how much the variance of a coefficient estimate is inflated due to its correlation with other predictors:

$$VIF_j = \frac{1}{1 - R_j^2}$$

When two variables are perfectly linearly dependent, R_j^2 approaches 1, which drives VIF toward infinity. In practice, R will throw an error rather than return an infinite value.

```r
tryCatch(car::vif(model_naive), error = function(e) cat("VIF Error:",
    conditionMessage(e), "\n"))
```

```
## VIF Error: there are aliased coefficients in the model
```

The error confirms what we already suspected. The model is rank-deficient and VIF cannot even be computed.

2.2 Rank of the Design Matrix

```r
X_naive <- model.matrix(~AdminArea + Province, data = df)
cat("Number of columns in X:", ncol(X_naive), "\n")
```

```
## Number of columns in X: 11
```

```r
cat("Rank of X:", qr(X_naive)$rank, "\n")
```

```
## Rank of X: 10
```

```r
cat("Rank deficiency:", ncol(X_naive) - qr(X_naive)$rank,
    "\n")
```

```
## Rank deficiency: 1
```

The rank deficiency tells you exactly how many columns are redundant. One column can be expressed as a perfect linear combination of the others, confirming that AdminArea and Province cannot coexist in the same model without a structural fix.

3 The Solution

The fix has two steps. First, we create a BASE reference category by merging one representative province from each region. Second, we apply sum-to-zero contrasts to both AdminArea and the recoded province variable.

3.1 Step 1: Choosing the BASE Province

Instead of arbitrarily picking a province to merge into BASE, we select it using a principled criterion: the province with the most observations in each region.

```r
base_provinces <- df |>
    count(AdminArea, Province) |>
    group_by(AdminArea) |>
    slice_max(n, n = 1) |>
    ungroup()

base_provinces
```

```
## # A tibble: 2 x 3
##   AdminArea     Province      n
##   <fct>         <fct>     <int>
## 1 Vlaams_Gewest Antwerpen    70
## 2 Waals_Gewest  Liege        84
```

```r
df$Province2 <- as.character(df$Province)

for (i in seq_len(nrow(base_provinces))) {
    area <- base_provinces$AdminArea[i]
    prov <- base_provinces$Province[i]
    df$Province2[df$AdminArea == area & df$Province == prov] <- "BASE"
}

lv <- c(setdiff(unique(df$Province2), "BASE"), "BASE")
df$Province2 <- factor(df$Province2, levels = lv)

df |>
    count(Province2) |>
    arrange(desc(n))
```

```
##         Province2   n
## 1            BASE 154
## 2         Hainaut  66
## 3  Vlaams_Brabant  65
## 4 Oost_Vlaanderen  65
## 5 West_Vlaanderen  64
## 6         Limburg  44
## 7      Luxembourg  44
## 8           Namur  37
## 9  Brabant_Wallon  27
```

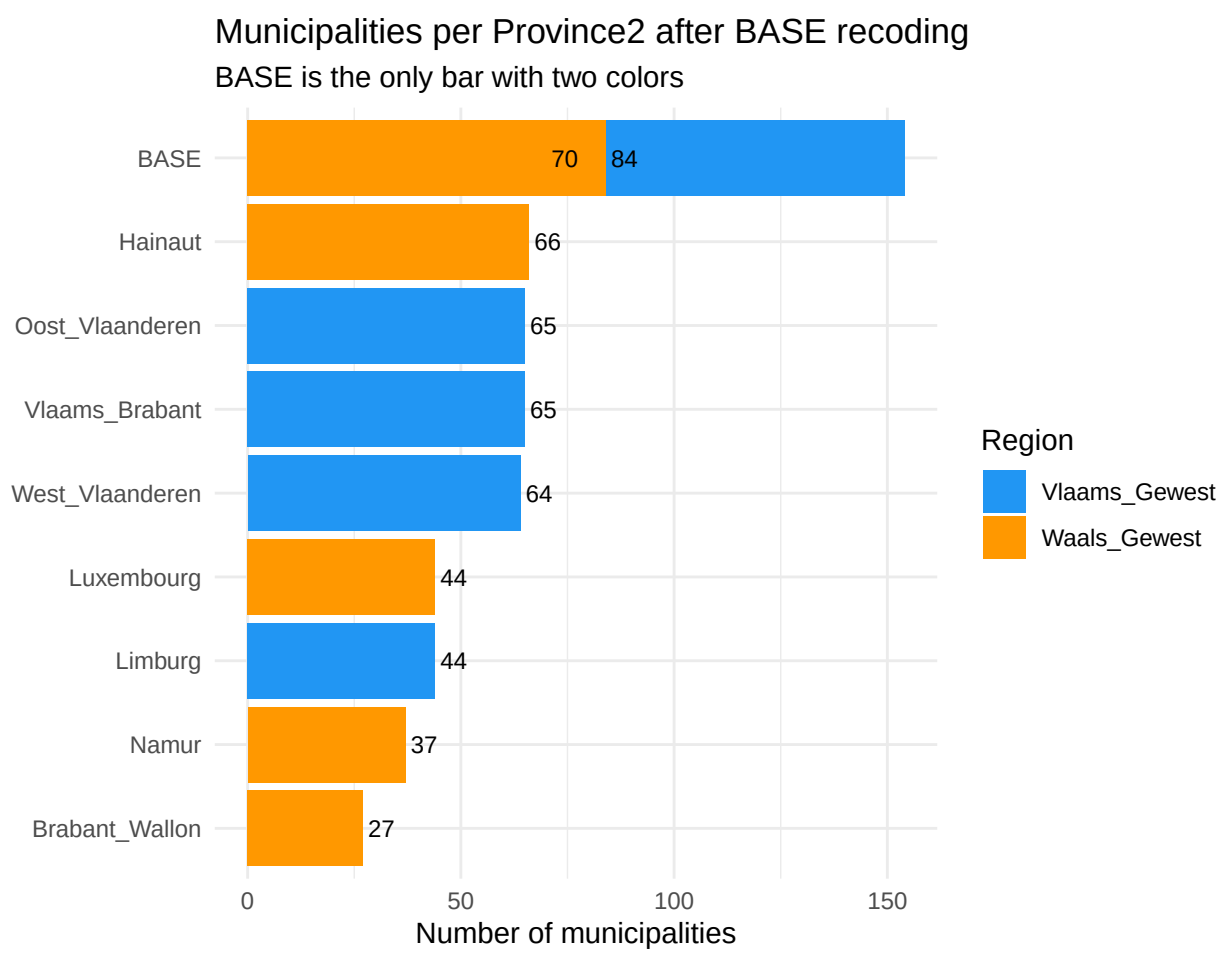
Notice that BASE now groups the most representative province from each region into a single reference category. This is what breaks the perfect linear dependence: BASE contains municipalities from both regions, so Province2 is no longer a perfect predictor of AdminArea.

3.2 Why Does This Work?

After the recoding, the BASE category contains municipalities from both regions, which means:

$$\text{Province2}_i = \text{BASE} \not\Rightarrow \text{AdminArea}_i \text{ is determined}$$

```
hierarchy2 <- df |>
    count(AdminArea, Province2) |>
    arrange(AdminArea, desc(n))

ggplot(hierarchy2, aes(x = reorder(Province2, n), y = n,
    fill = AdminArea)) + geom_col() + geom_text(aes(label = n),
    hjust = -0.2, size = 3) + coord_flip() + scale_fill_manual(values = c(Vlaams_Gewest = "#2196F3",
    Waals_Gewest = "#FF9800")) + labs(title = "Municipalities per Province2 after BASE recoding",
    subtitle = "BASE is the only bar with two colors", x = NULL,
    y = "Number of municipalities", fill = "Region") + theme_minimal(base_size = 11) +
    expand_limits(y = max(hierarchy2$n) * 1.15)
```



Compare this plot with the one in Section 1. The BASE bar is now split between two colors, which visually

confirms that the perfect separation has been broken.

3.3 Step 2: Applying Sum-to-Zero Contrasts

Sum-to-zero contrasts make each coefficient represent the deviation of that category from the overall mean:

$$\sum_{k=1}^{K} \beta_k = 0$$

```r
kA <- nlevels(df$AdminArea)
kP <- nlevels(df$Province2)

CA <- contr.sum(kA)
CP <- contr.sum(kP)

colnames(CA) <- levels(df$AdminArea)[1:(kA - 1)]
colnames(CP) <- levels(df$Province2)[1:(kP - 1)]

contrasts(df$AdminArea) <- CA
contrasts(df$Province2) <- CP
```

3.4 Fitting the Model

```r
df$Mean_income_c <- df$Mean_income - mean(df$Mean_income,
    na.rm = TRUE)

model_solution <- lm(med_price_house ~ AdminArea + Province2 +
    Mean_income_c, data = df)

summary(model_solution)
```

```
##
## Call:
## lm(formula = med_price_house ~ AdminArea + Province2 + Mean_income_c,
##     data = df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -72.828 -17.605  -0.276  14.715 105.490
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)              195.4839     1.1884 164.496  < 2e-16 ***
## AdminAreaVlaams_Gewest    24.2158     2.1045  11.506  < 2e-16 ***
## Province2Vlaams_Brabant   -1.6847     3.8649  -0.436  0.66308
## Province2West_Vlaanderen  -4.0536     3.8535  -1.052  0.29330
## Province2Oost_Vlaanderen -11.1386     3.6627  -3.041  0.00247 **
## Province2Limburg         -25.7090     4.2344  -6.071 2.35e-09 ***
## Province2Brabant_Wallon   52.2938     5.3002   9.866  < 2e-16 ***
## Province2Hainaut         -11.7501     3.6982  -3.177  0.00157 **
## Province2Luxembourg       -9.4124     4.1143  -2.288  0.02253 *
## Province2Namur             2.0581     4.3885   0.469  0.63927
## Mean_income_c              7.8208     0.3705  21.108  < 2e-16 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 25.37 on 555 degrees of freedom
## Multiple R-squared:  0.7756, Adjusted R-squared:  0.7716
## F-statistic: 191.8 on 10 and 555 DF,  p-value: < 2.2e-16
```

4 Verification: Did the Solution Work?

4.1 VIF After the Fix

```r
car::vif(model_solution)
```

```
##                   GVIF Df GVIF^(1/(2*Df))
## AdminArea     3.863738  1        1.965639
## Province2     5.785514  8        1.115954
## Mean_income_c 1.739086  1        1.318744
```

4.2 Rank of the New Design Matrix

```r
X_solution <- model.matrix(~AdminArea + Province2 + Mean_income_c,
    data = df)
cat("Number of columns in X:", ncol(X_solution), "\n")
```

```
## Number of columns in X: 11
```

```r
cat("Rank of X:", qr(X_solution)$rank, "\n")
```

```
## Rank of X: 11
```

```r
cat("Rank deficiency:", ncol(X_solution) - qr(X_solution)$rank,
    "\n")
```

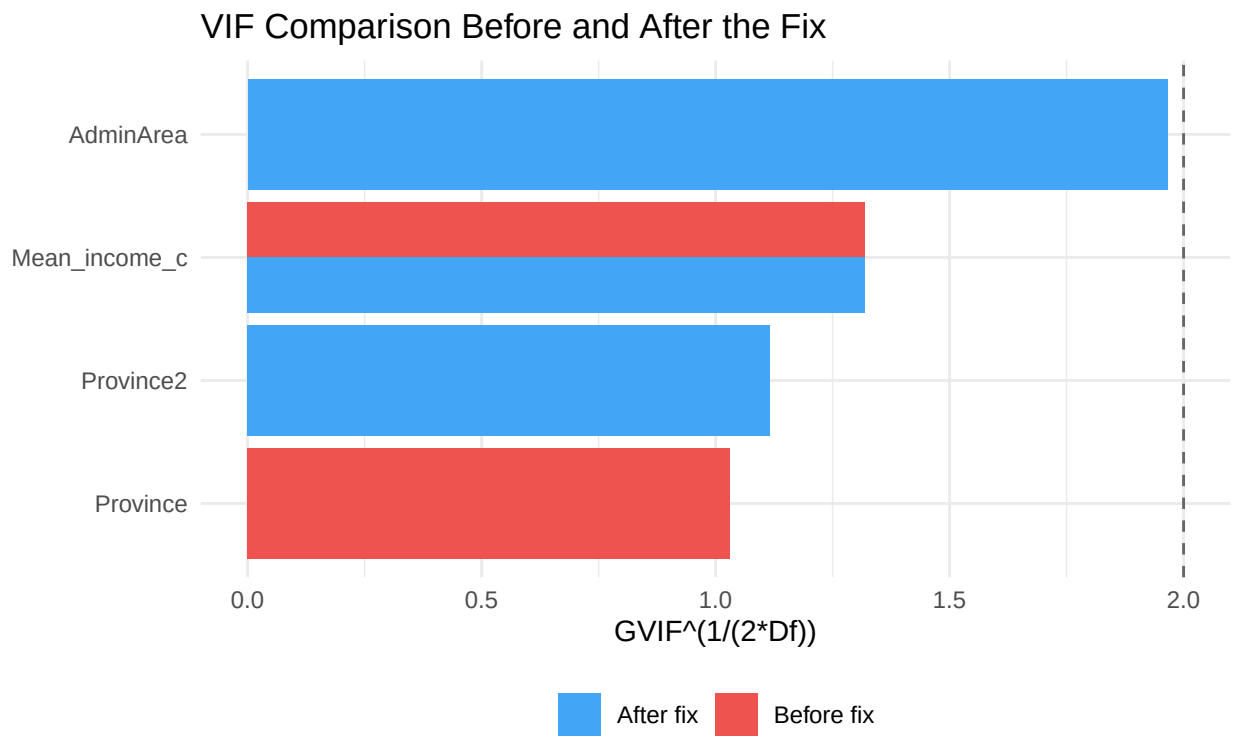
```
## Rank deficiency: 0
```

A rank deficiency of zero confirms that the design matrix now has full column rank. Every coefficient can be estimated without ambiguity.

4.3 Visual Comparison of VIF

```r
model_partial <- lm(med_price_house ~ Province + Mean_income_c,
    data = df)

vif_before <- car::vif(model_partial)
vif_after <- car::vif(model_solution)

vif_before_df <- data.frame(variable = names(vif_before[,
    3]), vif = vif_before[, 3], model = "Before fix")

vif_after_df <- data.frame(variable = names(vif_after[,
    3]), vif = vif_after[, 3], model = "After fix")

vif_combined <- bind_rows(vif_before_df, vif_after_df)

ggplot(vif_combined, aes(x = reorder(variable, vif), y = vif,
```

```r
    fill = model)) + geom_col(position = "dodge") + geom_hline(yintercept = 2,
    linetype = "dashed", color = "gray40") + coord_flip() +
    scale_fill_manual(values = c(`Before fix` = "#EF5350",
        `After fix` = "#42A5F5")) + labs(title = "VIF Comparison Before and After the Fix",
    x = NULL, y = "GVIF^(1/(2*Df))", fill = NULL) + theme_minimal(base_size = 11) +
    theme(legend.position = "bottom")
```



All VIF values in the fixed model are well below the threshold of concern. The solution not only allows the model to run but also produces coefficient estimates with acceptable levels of collinearity.

5 Interpretation: Reading Coefficients Under Sum-to-Zero Contrasts

5.1 The Intercept

Under sum-to-zero contrasts, the intercept represents the overall mean of the response variable when all continuous predictors are at their mean values.

```r
cat("Intercept (global mean housing price):", round(coef(model_solution)["(Intercept)"],
    2), "thousands\n")
```

```
## Intercept (global mean housing price): 195.48 thousands
```

5.2 Province Coefficients

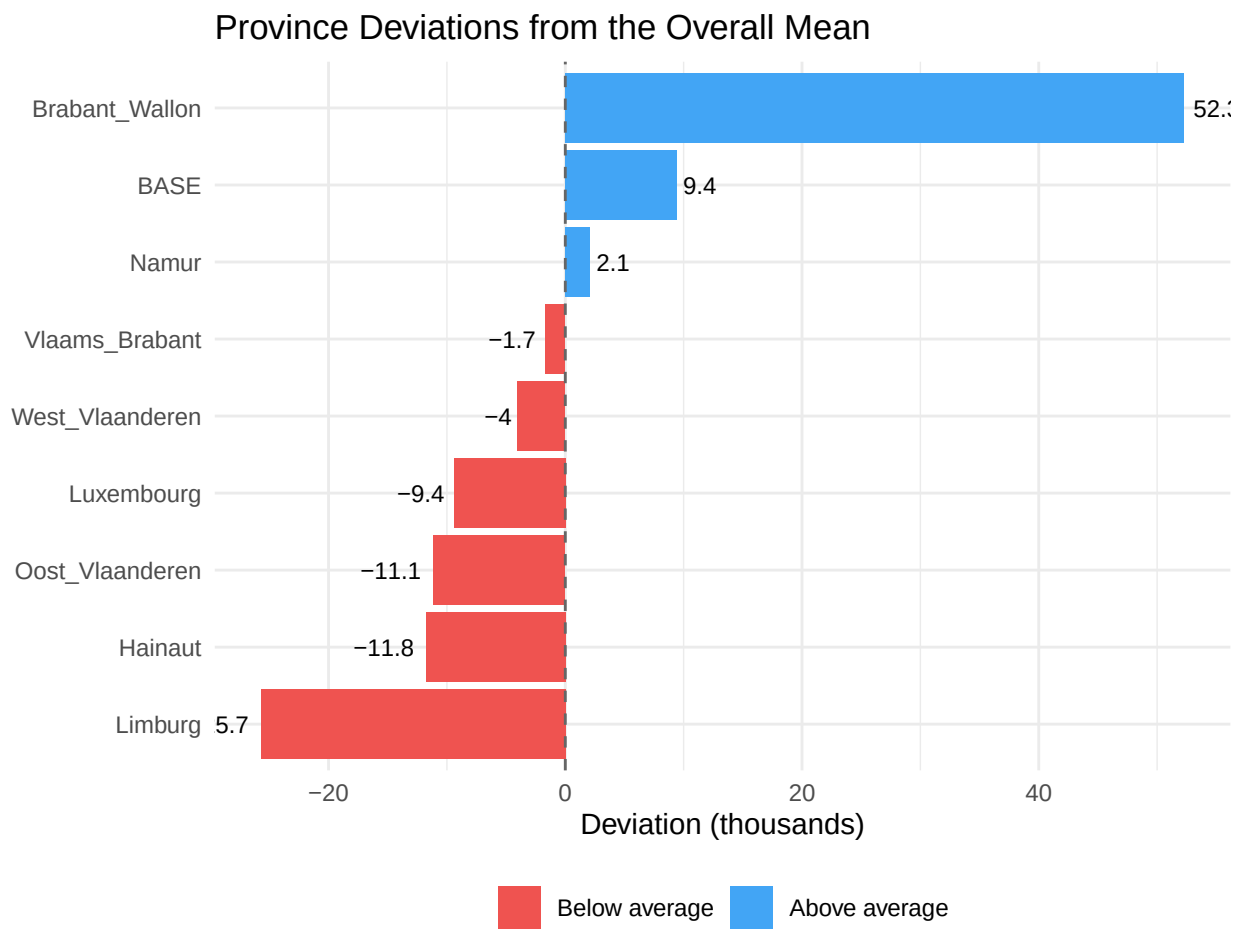
Each coefficient represents the deviation of that province from the overall mean. The implicit BASE coefficient is recovered as:

$$\beta_{\text{BASE}} = -\sum_{k \neq \text{BASE}} \beta_k$$

```r
province_coefs <- coef(model_solution)[grep("Province2",
    names(coef(model_solution)))]
beta_base <- -sum(province_coefs)
all_province_coefs <- c(province_coefs, BASE = beta_base)

comparison <- data.frame(Province = gsub("Province2", "",
    names(all_province_coefs)), Coefficient = round(all_province_coefs,
    2)) |>
    arrange(desc(Coefficient))

comparison
```

```
##                                Province Coefficient
## Province2Brabant_Wallon   Brabant_Wallon       52.29
## BASE                                BASE        9.40
## Province2Namur                     Namur        2.06
## Province2Vlaams_Brabant   Vlaams_Brabant       -1.68
## Province2West_Vlaanderen West_Vlaanderen       -4.05
## Province2Luxembourg           Luxembourg       -9.41
## Province2Oost_Vlaanderen Oost_Vlaanderen      -11.14
## Province2Hainaut                 Hainaut      -11.75
## Province2Limburg                 Limburg      -25.71
```

```r
ggplot(comparison, aes(x = reorder(Province, Coefficient),
    y = Coefficient, fill = Coefficient > 0)) + geom_col() +
    geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
    geom_text(aes(label = round(Coefficient, 1)), hjust = ifelse(comparison$Coefficient >
        0, -0.2, 1.2), size = 3) + coord_flip() + scale_fill_manual(values = c(`TRUE` = "#42A5F5",
    `FALSE` = "#EF5350"), labels = c(`TRUE` = "Above average",
    `FALSE` = "Below average")) + labs(title = "Province Deviations from the Overall Mean",
    x = NULL, y = "Deviation (thousands)", fill = NULL) +
    theme_minimal(base_size = 11) + theme(legend.position = "bottom")
```



5.3 A Concrete Example

To make this tangible, let us compare Vlaams_Brabant and Hainaut, which sit at opposite ends of the price spectrum.

```
beta_vb <- coef(model_solution)["Province2Vlaams_Brabant"]
beta_ha <- coef(model_solution)["Province2Hainaut"]
diff <- beta_vb - beta_ha

cat("Vlaams_Brabant coefficient:", round(beta_vb, 2), "\n")
```

```
## Vlaams_Brabant coefficient: -1.68
```

```
cat("Hainaut coefficient:", round(beta_ha, 2), "\n")
```

```
## Hainaut coefficient: -11.75
```

```
cat("Expected price difference:", round(diff, 2), "thousands\n")
```

```
## Expected price difference: 10.07 thousands
```

A municipality in Vlaams_Brabant is expected to have a housing price approximately that many thousands higher than a comparable municipality in Hainaut, holding mean income constant.

6 Applicability: Where Else Does This Problem Appear?

The problem documented in this tutorial is not specific to Belgium or to housing prices. It appears in any dataset where two categorical variables share a strict hierarchical relationship.

Latin America is particularly rich in nested geographic structures. In Ecuador, parishes are nested within cantons, cantons within provinces, and provinces within regions (Costa, Sierra, Amazonia, Insular). The same applies to municipalities nested within departments in Colombia and Peru, and to districts nested within cities in any urban study.

Beyond geography, the same structure appears in education research when schools are nested within districts, in clinical studies when patients are nested within hospitals, and in business analytics when products are nested within categories.

6.1 The General Recipe

Regardless of the specific context, the solution always follows the same steps:

1. Identify the two hierarchical levels that cause the problem
2. Choose a BASE category using a principled criterion such as the most frequent group at the lower level within each upper level group
3. Recode the lower level variable replacing the chosen categories with BASE
4. Set BASE as the last factor level
5. Apply contr.sum to both variables
6. Verify with VIF that the rank deficiency is resolved

6.2 Summary

This document presents a methodological solution for perfect multicollinearity caused by nested geographic variables. The solution combines BASE category recoding with sum-to-zero contrasts, producing coefficients that are directly interpretable as deviations from the global mean. This approach was developed during the course Applied Statistical Models I at ESPOL, Ecuador, as part of a group project. The methodological contribution documented here was developed independently by the author.